

GWDC Korea Hackathon: TRON Challenge

Brief

Challenge A: Build a Multi-Vendor TRON Energy Rental Procurement and Cost-Comparison Assistant

Build a procurement decision prototype that connects to multiple TRON Energy rental providers. It should fetch and normalize quotes and supply terms across providers based on the required Energy amount, rental duration, recipient address, budget, and latest delivery time. From the connected providers and their current quotes, it should compare single-vendor purchase vs. multi-vendor split orders, find a low-cost path that meets the requirements, and show the full fee breakdown and selection rationale.

Acceptance Criteria

- **Multi-Vendor Integration:** Integrate real quote sources from at least two providers. Show quote sources, normalize pricing units and display conventions, and list each provider's rental duration, available Energy amount, minimum order, and estimated delivery time.
- **Plan Comparison:** For a given request, compare single-vendor vs. split-order purchases. Explain the calculation method and search scope, and provide a recommended plan plus alternatives that meet the constraints. If no feasible plan exists, explain why.
- **Full Cost Breakdown:** Itemize rental fees and any applicable service fees, on-chain fees, and other costs.

Challenge B: Build an AI Asset Allocation and Yield Planning Assistant for TRON

Build an AI asset allocation and yield planning prototype for TRON ecosystem users. It should discover and compare suitable ways to deploy capital based on the user's holdings, time horizon, liquidity needs, and risk preferences. The system should integrate product data from ecosystem projects such as JustLend and USDD, use natural language to help users understand participation requirements, yield composition, operating costs, and risks, and generate evidence-based allocation plans. After the user confirms and completes any required approvals, it should help execute deposits, redemptions, or rebalances, and continue tracking positions and yield. The project should demonstrate the full flow from needs analysis, opportunity screening, plan comparison, and execution through position management and review.

Acceptance Criteria

- **Needs Analysis:** Capture holdings, horizon, liquidity needs, and risk preferences through conversation. Output a needs summary for the user to confirm, and proactively ask follow-up questions when information is missing.
- **TRON Ecosystem Integration:** Cover at least two TRON ecosystem projects: JustLend and USDD. Label data sources, update times, and applicable terms. Joint ecosystem campaigns (e.g., the TRON Carnival on Binance Wallet) may be included, but the campaign period, eligibility, and reward rules must be verified.
- **Plans & Yield Estimates:** For a given amount and horizon, provide at least two viable plans. Each should specify allocation amounts, yield composition, operating costs, exit conditions, and risks, and should separate base yield from incentive rewards.
- **AI Execution & Management:** Explain the allocation rationale. Turn the plan into executable deposit, redemption, or rebalance actions, and initiate execution only after user confirmation. As yields, liquidity needs, or user-defined conditions change, monitor positions, propose adjustments, and help execute them. Before each action, show the amount, fees, risks, and approval scope. After each action, log the result.
- **Tracking & Review:** Record the original plan and the assumptions behind the estimates. Show later changes in assets or yield, and compare expected vs. actual. Clearly labeled historical replay or simulated positions may be used to demonstrate the review flow.

Challenge C: Build a GasFree Stablecoin Batch Payment and Reconciliation Assistant

Build a stablecoin payment prototype for teams and merchants. Users import payee addresses, amounts, and business notes, then complete payments via GasFree without holding TRX. Before sending, the assistant should check the address format, duplicates, available balance, and estimated fees. After the user confirms and signs, it should orchestrate the payments, track each transaction, and reconcile results against the original list.

Acceptance Criteria

- **Import & Validation:** Support CSV or spreadsheet import. Flag invalid addresses, unexpected amounts, and suspected duplicates. Show payable items, exceptions, and totals. Allow users to correct or confirm.
- **Balance & Fee Pre-Check:** Show the GasFree payer account, supported tokens, available balance, and estimated total fees.
- **Authorization & Orchestration:** Once the user confirms the payee list and fees, collect the required wallet signatures and submit the payments.
- **Per-Transaction Status & Recovery:** Distinguish statuses such as pending submission, processing, success, and failure. Record request IDs and on-chain transaction hashes, and show failure reasons. On timeout, verify status first; retries must not cause duplicate payments.
- **Reconciliation & Export:** Link each result to its original row and business note. Export the amount, status, transaction hash, and actual fees when available. Summarize succeeded, failed, and awaiting-confirmation items.